SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I **D.Naveen / 192372177** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

1. Secure Password Storage in Web Applications
A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.
B. Suggest a secure method for storing passwords and justify your choice.

2. Digital Signature for Document Authentication
A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. Explain how digital signatures can be used in this scenario.
B. Justify how this method ensures both integrity and non-repudiation.

3. Preventing Replay Attacks in Online Transactions
An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

A. Explain how replay attacks occur.
B. Suggest a cryptographic technique to prevent such attacks and justify your answer.

4. Secure Communication over Public Wi-Fi
A user accesses sensitive information using a public Wi-Fi network without encryption.

A. Identify the risks involved in this scenario.
B. Recommend a secure communication method and justify how it protects user data.

5. Data Integrity in Cloud Storage
An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

A. Explain how hashing can be used to verify data integrity.
B. Suggest an additional mechanism to enhance trust and justify your answer.

1. Secure Password Storage in Web Applications

A. Why storing plaintext passwords is insecure

- **Total exposure on breach:** If the database is leaked through SQL injection, a stolen backup or an insider, every user's password is immediately readable. No cracking effort is needed.
- **Credential reuse:** Users reuse passwords across sites, so one breach leads to account takeover (credential stuffing) on email, banking and social media.
- **Insider threat:** Administrators and developers with database access can read and misuse passwords, which breaks confidentiality and accountability.
- **Legal and reputational damage:** It violates standards such as OWASP, ISO 27001, PCI-DSS and data-protection laws, and it destroys user trust.

B. Secure method and justification

Recommended method: salted, slow, adaptive password hashing (Argon2id, bcrypt or PBKDF2) with an optional pepper.

- **Process:** At registration the server generates a unique random salt (at least 16 bytes) and computes *hash = KDF(password || salt, cost factor)*. It stores only *(salt, hash, cost parameters)*. At login it recomputes the hash with the stored salt and compares in constant time.
- **One-way function:** A hash cannot be reversed, so a stolen database does not reveal the passwords directly.
- **Unique salt:** Identical passwords produce different hashes. This defeats rainbow tables and precomputed lookup attacks, and forces the attacker to crack each user separately.
- **Slow, tunable cost:** bcrypt, Argon2id (memory-hard) and PBKDF2 (many iterations) make each guess expensive and resist GPU/ASIC brute force. The cost can be raised as hardware improves.
- **Why not plain SHA-256 or MD5:** General-purpose hashes are very fast, which lets an attacker test billions of guesses per second.
- **Extra hardening:** A secret pepper kept outside the database (in an HSM or vault), password-strength rules, login rate-limiting and multi-factor authentication.

2. Digital Signature for Document Authentication

A. How digital signatures are used

A digital signature uses public-key cryptography (for example RSA) together with a hash function.

- **Key setup:** The company owns an RSA key pair. The private key is kept secret and the public key is published in a certificate issued by a trusted Certificate Authority.
- **Signing (sender):** (1) Compute the digest $H = SHA\text{-}256(document)$. (2) Encrypt the digest with the company's private key: $Signature = H^d \bmod n$. (3) Send the document together with the signature and certificate.
- **Verification (receiver):** (1) Recompute $H' = SHA\text{-}256(received\ document)$. (2) Decrypt the signature with the public key: $H = Signature^e \bmod n$. (3) If $H = H'$, the document is accepted as authentic and unaltered. Otherwise it is rejected.
- The receiver also checks that the certificate is valid and has not been revoked.

B. How it ensures integrity and non-repudiation

- **Integrity:** The signature is computed over the hash of the document. Changing even one bit changes the digest completely (avalanche effect), so the recomputed hash will not match the signed hash and tampering is detected.
- **Authentication:** Only the holder of the private key can produce a signature that verifies with the matching public key. This proves the document came from the company.
- **Non-repudiation:** Because the private key is held only by the signer, the company cannot later deny having signed. A third party, such as a court, can verify the signature using the public key and certificate. A symmetric MAC cannot offer this, because both sides share the key.
- **Efficiency:** Signing the short hash instead of the whole document keeps RSA operations fast.

3. Preventing Replay Attacks in Online Transactions

A. How replay attacks occur

- **Capture:** The attacker sniffs a valid, correctly authenticated transaction message on the network (for example "Pay Rs. 5,000 to X" with its MAC, token or signature) through a man-in-the-middle position or a compromised network.
- **Resend:** The attacker does not need to break the encryption or know the key. They simply retransmit the identical message later.
- **Acceptance:** The server finds the message cryptographically valid and has no way to tell it is old, so it processes the payment again. This causes duplicate debits, repeated fund transfers or replayed login tokens.

B. Technique to prevent it and justification

Recommended technique: freshness using nonces and timestamps, bound to the message by a MAC or digital signature, inside a secure channel (TLS with per-session keys).

- **Message format:** Each transaction carries *(transaction data, unique nonce or sequence number, timestamp)* and a tag *HMAC-SHA256(K, data* || *nonce* || *timestamp)*, or a digital signature over the same fields.
- **Server check:** The server rejects any message whose timestamp is outside a short validity window (for example 30-60 seconds) or whose nonce or sequence number has already been used. It keeps a cache of recent nonces.
- **Challenge-response option:** The server sends a fresh random challenge for each transaction and the client must include it in its signed or MACed reply. A recorded reply is useless against a new challenge.
- **Why it works:** The nonce and timestamp are covered by the MAC or signature, so the attacker cannot change them without invalidating the tag. A replayed message either carries a used nonce or has expired.
- **Defence in depth:** TLS gives every session fresh keys and sequence numbers, which stops cross-session replay. One-time transaction tokens (OTP) add another layer.

4. Secure Communication over Public Wi-Fi

A. Risks involved

- **Eavesdropping (sniffing):** On an open network, anyone nearby can capture traffic with simple tools. Passwords, banking details and personal data sent in clear text are exposed

(loss of confidentiality).

- **Man-in-the-middle attacks:** An attacker can sit between the user and the server, then read or modify data and inject malicious content.
- **Evil twin / rogue access point:** A fake hotspot with a genuine-looking name lures users to connect and routes all their traffic through the attacker.
- **Session hijacking and DNS spoofing:** Stolen cookies or tampered DNS replies take over accounts or redirect users to phishing sites.
- **Other risks:** SSL stripping, malware injection and attacks from other devices on the same network.

B. Recommended method and justification

Recommended method: use a VPN (IPsec, OpenVPN or WireGuard) and access sites only over HTTPS (TLS).

- **Key exchange:** The client and server authenticate each other with certificates (RSA/ECDSA signatures) and agree on a shared secret using Diffie-Hellman or ECDHE. The key is never sent over the network, so an eavesdropper cannot obtain it.
- **Confidentiality:** Traffic is encrypted with a symmetric cipher such as AES-256-GCM or ChaCha20, so captured packets are unreadable on the Wi-Fi network.
- **Integrity and authentication:** AEAD or HMAC tags detect modification. Certificate validation prevents impersonation by evil-twin hotspots and man-in-the-middle attackers.
- **Forward secrecy:** Ephemeral DH keys give each session a fresh key, so a later key compromise does not expose past sessions.
- **Good practice:** Turn off auto-connect, verify the network name, use WPA2/WPA3 where available, keep devices updated and use multi-factor authentication.

5. Data Integrity in Cloud Storage

A. Using hashing to verify integrity

- **Before upload:** Compute a cryptographic hash (such as SHA-256) of each file and store this digest securely, either locally or in a separate trusted location. *Digest = SHA-256(file)*.
- **After download or periodic audit:** Recompute the hash of the file retrieved from the cloud and compare it with the stored digest.
- **Result:** If the digests match, the data is unchanged. If they differ, the data was altered by corruption, a malicious insider or an attacker.
- **Why it works:** SHA-256 is one-way, collision-resistant and has the avalanche property, so any change, even one bit, produces a completely different digest and an attacker cannot craft a modified file with the same hash.
- For large data sets, hashes of blocks can be combined into a Merkle tree so that integrity can be checked efficiently.

B. Additional mechanism to enhance trust

Recommended mechanism: digital signatures (or HMAC) on the hash, with signed timestamps and secure key management.

- **Problem with a bare hash:** If an attacker can modify the file, they can also replace the stored hash with a new one, so a plain hash alone gives no authenticity.
- **Solution:** The data owner signs the digest with their private key: *Sig = Sign(SK, SHA-256(file))*. Anyone can verify it with the public key. The attacker cannot forge a valid

signature without the private key. An HMAC with a secret key gives the same protection for a single trusted party.

- **Benefits:** It provides integrity, authenticity of the data owner and non-repudiation. A trusted timestamp also proves when the data was stored.
- **Supporting controls:** Encrypt data at rest (AES-256) for confidentiality, keep versioned backups and audit logs, and protect keys in an HSM or KMS.

Conclusion: Hashing, salting, digital signatures, nonces and key exchange each address a different security goal (confidentiality, integrity, authentication and non-repudiation). Applying the right primitive to each scenario strengthens overall information security.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand